Database Initialization & Architecture Guide

PostgreSQL Guest House Booking System — Relational Order, Tiered Dependencies & Data Requirements

27 TABLES | 5 DEPENDENCY TIERS | FOREIGN KEY SAFE

To populate the PostgreSQL guest house database without encountering foreign key integrity violations (`FOREIGN KEY constraint failed`), initial data seeding must follow a strict **5-tier topological sequence**.

[Tier 1: Independent Masters] (Hostels, App Settings, Mail Templates, Guest Houses, Whitelist, Invoice Counters)
↓
[Tier 2: Core Identity & Setup] (Units, Profiles, Rooms, Rate Limits, Form Configs)
↓
[Tier 3: Domain Rules & User Metadata] (Academic Records, MFA, Sessions, Projects, Tariffs, Room Blocks)
↓
[Tier 4: Transactional Core] (Bookings)
↓
[Tier 5: Dependent Execution & Invoicing] (Booking Rooms, Guests, Logs, Room Holds, Outbox, Invoices, Security Audit)

1. Executive Summary Table of Initialization Tiers

Tier	Table Name	Primary / Foreign Keys	Purpose & Data Guidelines
Tier 1	**hostels**	`PK: name (text)`	Campus hostels lookup table (e.g., Malhar, Saveri, Kalyani).
Tier 1	**app_settings**	`PK: key (text)`	Key-value system rules (JSONB value: booking caps, time windows).
Tier 1	**mail_templates**	`PK: event_key (text)`	Notification text and headers for booking system mail events.
Tier 1	**official_email_whitelist**	`PK: email (text)`	Approved emails permitted for submitting official stay requests.
Tier 1	**guest_houses**	`PK: id (uuid)`	Physical guest house facilities (Bageshri, Hamsanandi) with room counts.
Tier 1	**invoice_counters**	`PK: fy (text)`	Tracks sequence numbers atomically per financial year (e.g., 2026-27).
Tier 2	**units**	`PK: id (uuid)` `FK: head_id → profiles`	Departments, clubs, councils, offices. Drives approval routing.
Tier 2	**profiles**	`PK: id (uuid)` `FK: unit_id, hostel_name`	User accounts, roles, credentials, department assignments.
Tier 2	**rooms**	`PK: id (uuid)` `FK: guest_house_id`	Physical guest rooms mapped to guest houses. Set active status.
Tier 2	**rate_limits**	`PK: key (text)`	Sliding window rate limit counters per IP/ user key.

Tier	Table Name	Primary / Foreign Keys	Purpose & Data Guidelines
Tier 2	**form_configs**	`PK: role (user_role)`	Role-specific form field configurations (JSONB).
Tier 3	**academic_records**	`PK: id (uuid)` `FK: imported_by → profiles`	Imported institute master records for identity verification.
Tier 3	**user_mfa**	`PK/FK: user_id → profiles`	Encrypted 2FA secrets and recovery codes per profile.
Tier 3	**sessions**	`PK: id (uuid)` `FK: user_id → profiles`	Server-side session tokens, timestamps, IP address logs.
Tier 3	**privacy_requests**	`PK: id (uuid)` `FK: user_id, handled_by`	DPDP data export or erasure compliance requests.
Tier 3	**user_import_batches**	`PK: id (uuid)` `FK: actor_id, reverted_by`	Spreadsheet user import audit records for rollback operations.
Tier 3	**projects**	`PK: id (uuid)` `FK: created_by → profiles`	Research grant projects for official debit head billing.
Tier 3	**tariffs**	`PK: id (uuid)` `FK: guest_house_id, created_by`	Pricing schedule per room type, stay type, and meal choice.
Tier 3	**room_blocks**	`PK: id (uuid)` `FK: room_id → rooms`	Maintenance blocks (tstzrange) preventing room allocation.
Tier 4	**bookings**	`PK: id (uuid)` `FK: user_id, guest_house_id, project_id`	Central booking request records holding stay details & status.
Tier 5	**booking_rooms**	`PK: id (uuid)` `FK: booking_id, assigned_room_id`	Logical room cards assigned to a specific booking.
Tier 5	**booking_guests**	`PK: id (uuid)` `FK: booking_id, booking_room_id`	Individual guest records, relationships, redacted ID numbers.
Tier 5	**booking_logs**	`PK: id (uuid)` `FK: booking_id, action_by`	Immutable audit log of all booking pipeline status transitions.
Tier 5	**room_holds**	`PK: id (uuid)` `FK: booking_id, room_id`	Exclusion constraint guard ranges preventing double booking.
Tier 5	**email_outbox**	`PK: id (uuid)` `FK: booking_id → bookings`	Transactional mail delivery queue with retry count & status.
Tier 5	**invoices**	`PK: id (uuid)` `FK: booking_id, paid_by, issued_by`	Billing invoice snapshot, GST calculation, payment tracking.
Tier 5	**security_audit**	`PK: id (bigint)` `FK: actor_id → profiles`	Immutable append-only audit trail for administrative changes.

2. Detailed Tier Specifications & Seeding Logic

Tier 1: Independent Standalone Masters

These tables have zero foreign key references to other tables in the schema. Initialize them first to establish valid lookups and system configuration values.

Tier 2: Identity & Base Organisational Hierarchy

Contains user profiles, organisational units, and physical room inventory.

> **Circular Dependency Resolution:** Table `units` references `profiles(head_id)`, while `profiles` references `units(unit_id)`. To seed cleanly:
> 1. Insert rows into `units` with `head_id = NULL`.
> 2. Insert rows into `profiles` referencing valid `unit_id` values.
> 3. Execute `UPDATE units SET head_id = ...` to link unit heads.

Tier 3: Domain Rules, Projects & User Metadata

Stores metadata dependent on established users and guest houses, including research projects, room blocking periods, tariffs, active user sessions, and 2FA credentials.

Tier 4: Transactional Core (Bookings)

Holds the central `bookings` table. Each booking requires a valid requester `user_id` (from `profiles`), a target `guest_house_id`, and an optional `project_id` for debiting.

Tier 5: Dependent Execution, Audit Logs & Invoicing

These child tables directly reference active bookings or depend on finalized transactions (such as room allocation, guest details, audit logs, and issued invoices).